

Intelligence Artificielle : Réseaux de neurones

– Perceptron –

François Buet
francois.buet@telecomnancy.eu

TELECOM Nancy

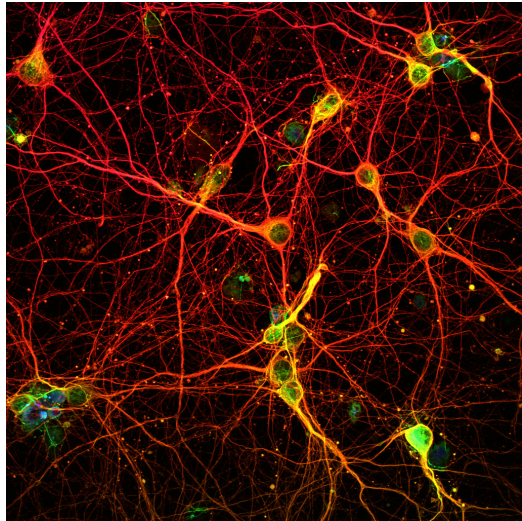
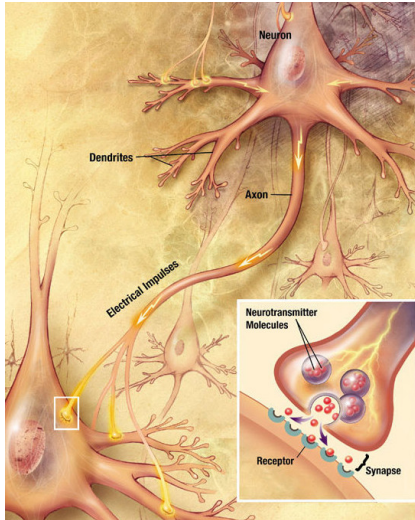
2025-2026



Références du cours

- Bhiksha Raj. *Introduction to neural networks*.
Lecture, Lisbon Machine Learning School, July 2023
- Alexandre Bérard. *Neural Machine Translation Architectures and Applications*.
Theses, Université de lille, June 2018

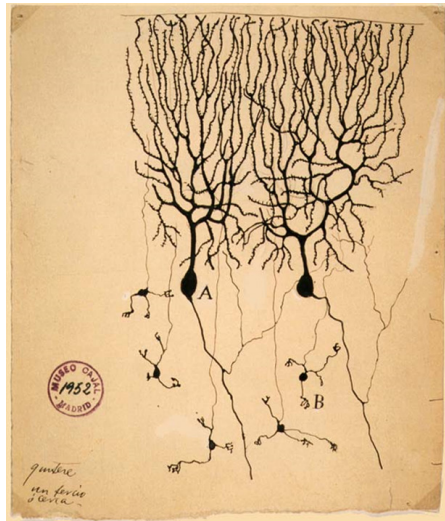
Le neurone biologique, unité fonctionnelle du système nerveux



À la fin du XIX^e siècle, l'invention du neurone...

- **1873** : coloration argentique pour visualiser le tissu nerveux (Camillo Golgi)
- **1888** : mise en évidence de cellules nerveuses (Santiago Ramón y Cajal)
- **1891** : introduction du mot « neurone » (Heinrich Wilhelm Waldeyer)

νεῦρον : « nerf, fibre »



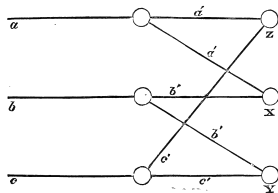
... et les précurseurs du connexionnisme

Bain (1873) : *Mind and Body*

CASE OF DIFFERENCE OF GROUPING. 111

branches of b , or a' b' in a cell X ; b' c' unite in Y ; a' c' in Z . These cells X , Y , Z , are supposed to be the commencement of motor fibres, each communicating with a separate muscular group, and rousing a distinctive movement. By this plan we comply with the primary condition of assigning a separate outcome to every different combination of sensory impressions.

Fig. 2.



We may compare this diagram with the following, given by Dr. Lionel Beale, to show the manner of junction of nerve fibres with caudate nerve cells. The crossing of fibres from one cell to collateral cells is exactly what is supposed in the foregoing representation. Dr. Beale is not advocating any theory of the physical basis of our in-

112 THE INTELLECT.

tellectual acquisitions; his object is to represent the connexions of fibres and corpuscles as actually exhibited. The conformity of his diagram with the scheme of cross connections required by the foregoing hypothetical scheme, is very striking. But, indeed, without a most extensive system of these lateral communications, we should be wholly unable to imagine the embodiment of our distinctive mental impressions.

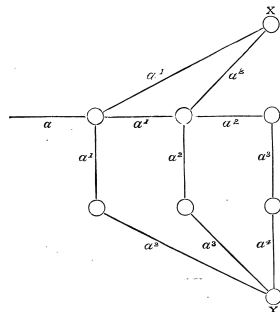
Fig. 3.



We have taken the simplest case possible—binary groupings of three elements, a , b , c . The diagram shews that for these we need three primary fibres, six branching fibres, and six cells. Our acquisitions involve far more complex groupings. To give a distinctive character to the most ordinary impression on the eye, or

ILLUSTRATION OF VARYING INTENSITY. 115

a^1 , is seen at the top of the figure proceeding with a second branching, a^2 , to the cell marked X . This con-



a^1 and a^2 give X
 a^3 , a^5 , and a^7 give Y .

vergence would represent the lowest degree of intensity. A higher degree of intensity makes a larger sweep,

Le connexionnisme, modélisation mathématique des phénomènes mentaux

McCulloch and Pitts (1943)

BULLETIN OF
MATHEMATICAL BIOPHYSICS
VOLUME 5, 1943

A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH AND WALTER PITTS

FROM THE UNIVERSITY OF ILLINOIS, COLLEGE OF MEDICINE,
DEPARTMENT OF PSYCHIATRY AT THE ILLINOIS NEUROPSYCHIATRIC INSTITUTE,
AND THE UNIVERSITY OF CHICAGO

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the calculus are discussed.

130 LOGICAL CALCULUS FOR NERVOUS ACTIVITY

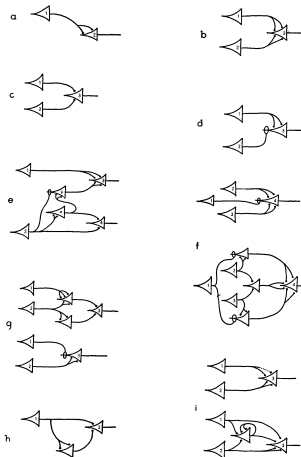


FIGURE 1

Le modèle de McCulloch et Pitts, neurone formel avec seuil d'activation

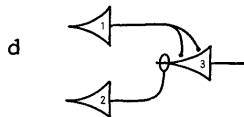
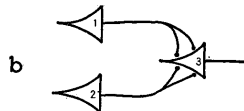
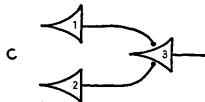
- Synapse excitatrice : —●
- Synapse inhibitrice : —○

- (a) $N_2 = N_1$
(b) $N_3 = N_1 \vee N_2$
(c) $N_3 = N_1 \wedge N_2$
(d) $N_3 = N_1 \wedge \neg N_2$

↪ Capable de réaliser des opérations booléennes

130

LOGICAL CALCULUS FOR NERVOUS ACTIVITY



Le Perceptron, neurone formel avec combinaison linéaire des entrées

Rosenblatt (1958)

Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN¹

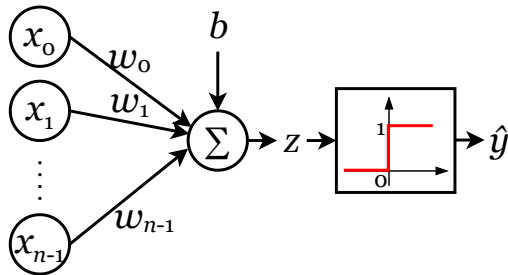
F. ROSENBLATT

Cornell Aeronautical Laboratory

If we are eventually to understand the capability of higher organisms for perceptual recognition, generalization, recall, and thinking, we must first have answers to three fundamental questions:

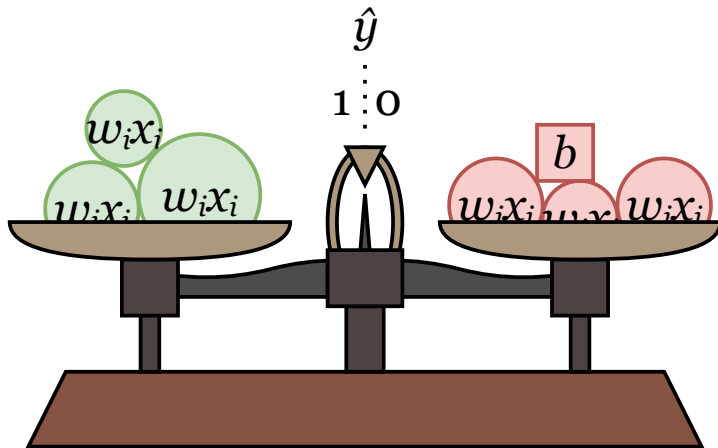
1. How is information about the physical world sensed, or detected, by the biological system?
2. In what form is information stored, or remembered?
3. How does information contained in storage, or in memory, influence recognition and behavior?

and the stored pattern. According to this hypothesis, if one understood the code or "wiring diagram" of the nervous system, one should, in principle, be able to discover exactly what an organism remembers by reconstructing the original sensory patterns from the "memory traces" which they have left, much as we might develop a photographic negative, or translate the pattern of electrical charges in the "memory" of a digital computer. This hypothesis is appealing in its simplicity and ready intelligibility, and a large family of theoretical brain



$$\hat{y} = \begin{cases} 1 & \text{si } z = \sum_i w_i x_i + b > 0 \\ 0 & \text{sinon} \end{cases}$$

Une première interprétation intuitive : modélisation d'une prise de décision



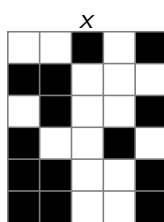
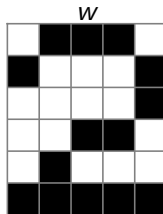
Une deuxième interprétation : le vecteur poids comme gabarit

$$w = \begin{bmatrix} w_0 \\ \vdots \\ w_{n-1} \end{bmatrix}, x = \begin{bmatrix} x_0 \\ \vdots \\ x_{n-1} \end{bmatrix}$$

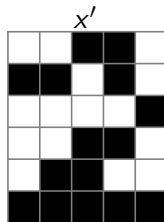
$$\sum_i w_i x_i + b > 0$$

$$w^\top x > -b$$

$$\cos(\widehat{w, x}) > -\frac{b}{\|w\| \|x\|}$$



$\hat{y} = 0$



$\hat{y}' = 1$

$\hookrightarrow$ Activation si x respecte la contrainte de proximité angulaire par rapport à w

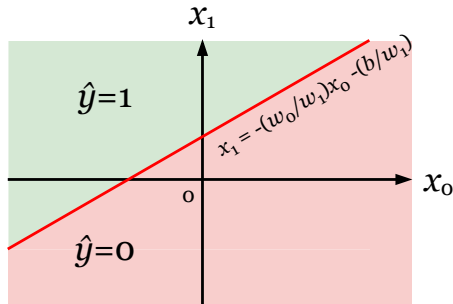
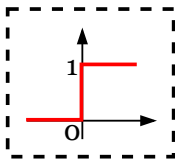
Le Perceptron, un classificateur linéaire

$\sum_i w_i x_i + b = 0$ définit un hyperplan qui divise $\mathcal{X}$ en deux : c'est la frontière de décision

Dans le cas $\dim(\mathcal{X}) = 2$:

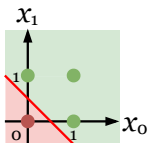
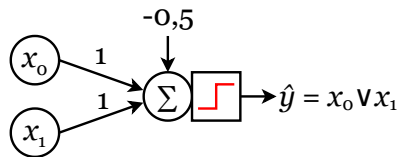
$$w_0 x_0 + w_1 x_1 + b > 0$$

$$x_1 > -\frac{w_0}{w_1} x_0 - \frac{b}{w_1}$$

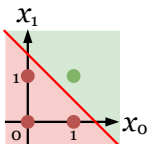
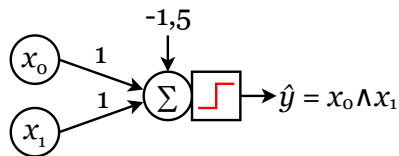


$\hookrightarrow$ Activation dans le demi-espace au dessus de la droite $x_1 = -\frac{w_0}{w_1} x_0 - \frac{b}{w_1}$

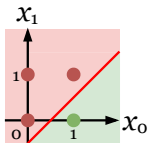
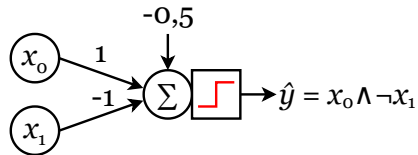
Reproduction des fonctions logiques



$$\hat{y} = 1 \text{ si } x_0 + x_1 > 0,5 \Leftrightarrow x_1 > -x_0 + 0,5$$

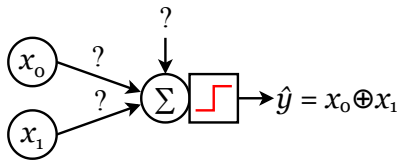


$$\hat{y} = 1 \text{ si } x_0 + x_1 > 1,5 \Leftrightarrow x_1 > -x_0 + 1,5$$

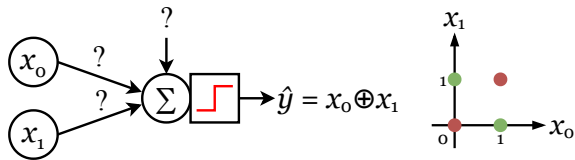


$$\hat{y} = 1 \text{ si } x_0 - x_1 > 0,5 \Leftrightarrow x_1 < x_0 - 0,5$$

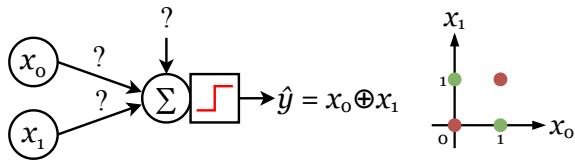
Reproduction du OU exclusif



Reproduction du OU exclusif

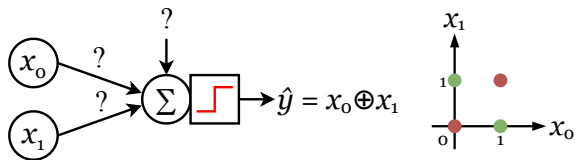


Reproduction du OU exclusif



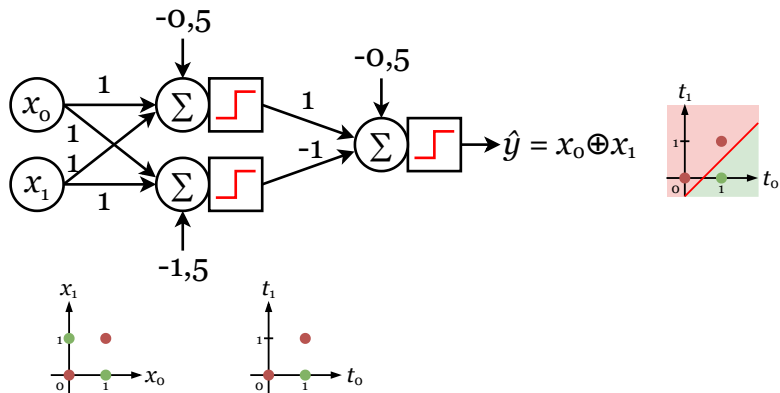
$\hookrightarrow$ *Impossible! Mais...*

Reproduction du OU exclusif



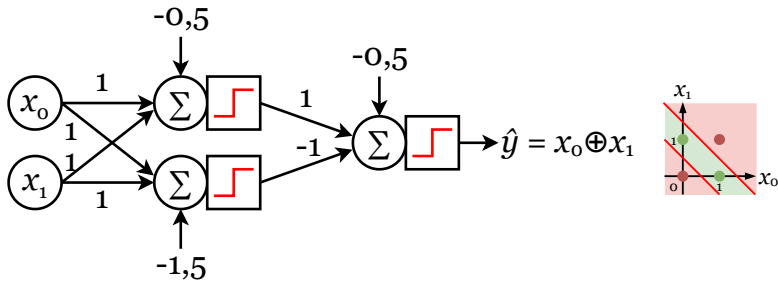
$\hookrightarrow$ Impossible! Mais... $x_0 \oplus x_1 = (x_0 \vee x_1) \wedge \neg(x_0 \wedge x_1)$

Reproduction du OU exclusif, avec un perceptron multicouche



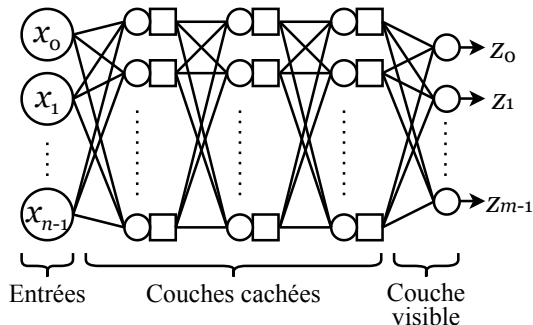
$$\begin{cases} t_0 &= x_0 \vee x_1 \\ t_1 &= x_0 \wedge x_1 \end{cases}, \quad \hat{y} = t_0 \wedge \neg t_1$$

Reproduction du OU exclusif, avec un perceptron multicouche



$$\left(\begin{array}{c} x_1 \\ \uparrow \\ \text{Green region} \\ \downarrow \\ x_0 \end{array} \right) \wedge \neg \left(\begin{array}{c} x_1 \\ \uparrow \\ \text{Red region} \\ \downarrow \\ x_0 \end{array} \right) = \left(\begin{array}{c} x_1 \\ \uparrow \\ \text{Green region} \\ \downarrow \\ x_0 \end{array} \right) \wedge \left(\begin{array}{c} x_1 \\ \uparrow \\ \text{Red region} \\ \downarrow \\ x_0 \end{array} \right) = \left(\begin{array}{c} x_1 \\ \uparrow \\ \text{Green region} \\ \downarrow \\ x_0 \end{array} \right)$$

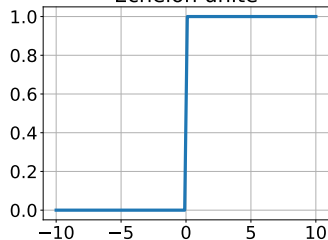
Le Perceptron multicouche, un modèle non linéaire



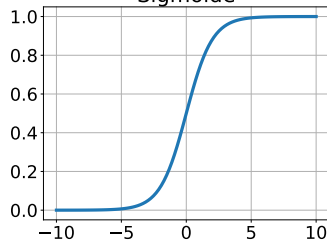
- La couche visible correspond au mode linéaire
- Chaque couche cachée est activée par une fonction non linéaire (sans quoi on reviendrait au cas monocouche : *une comb. lin. de comb. lin. est une comb. lin.*)
- Les couches cachées extraient des traits de plus en plus complexes, jusqu'à permettre le traitement par la couche visible

Les activations, fonctions non linéaires

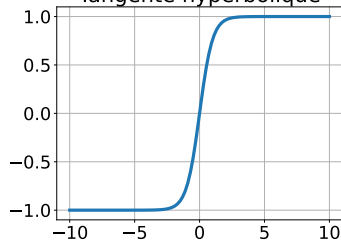
Échelon unité



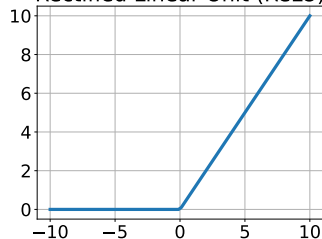
Sigmoïde



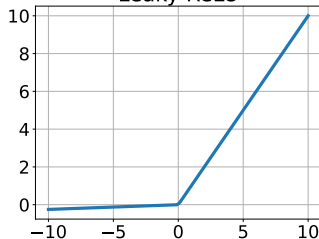
Tangente hyperbolique



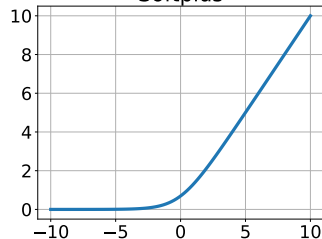
Rectified Linear Unit (ReLU)



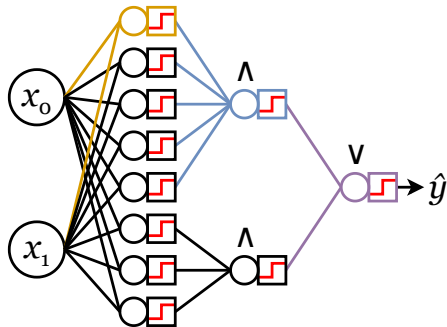
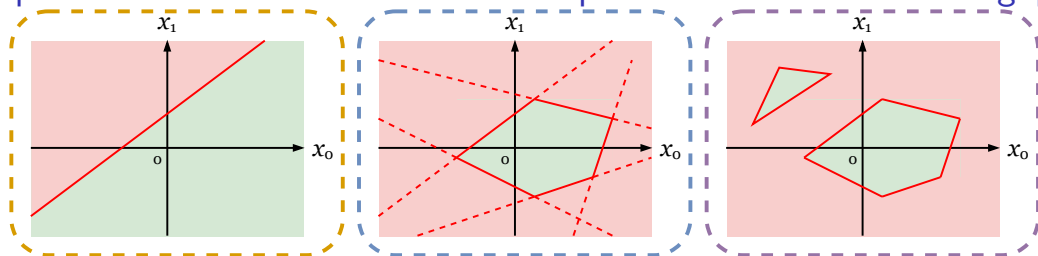
Leaky ReLU



Softplus

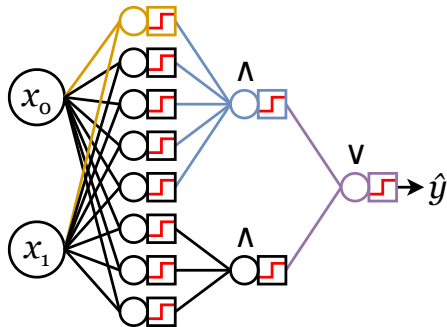
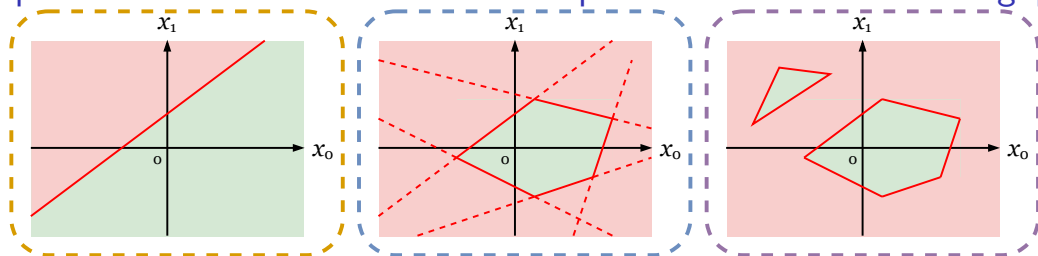


Composer des frontières de décision complexes avec les fonctions logiques



Le perceptron multicouche est un classificateur universel !

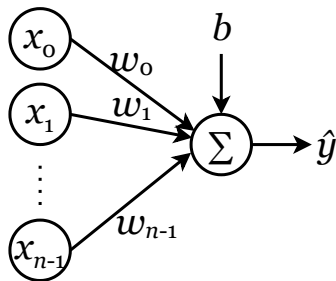
Composer des frontières de décision complexes avec les fonctions logiques



Le perceptron multicouche est un classificateur universel !

Et pour la régression ?

Perceptron pour la régression

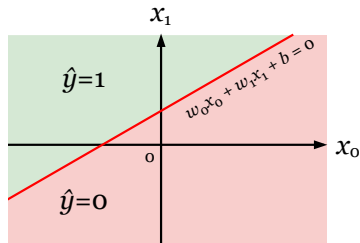
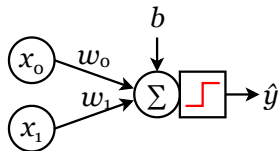


$$\hat{y} = \sum_i w_i x_i + b$$

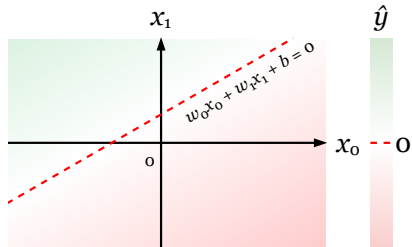
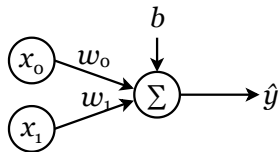
$\hookrightarrow$ Pas d'activation en sortie

Le Perceptron sans activation, un régresseur linéaire

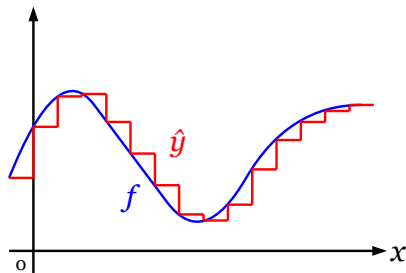
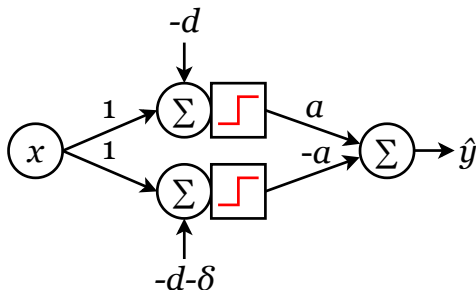
Classification :



Régression :



Modéliser une fonction à valeurs continues non linéaire, avec un MLP

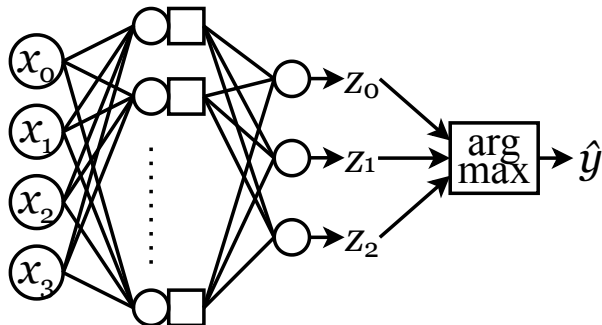


$$a \times \left(\begin{array}{c} \hat{y} \\ 1 \\ \hline 0 \quad d \end{array} \right) - a \times \left(\begin{array}{c} \hat{y} \\ 1 \\ \hline 0 \quad d+\delta \end{array} \right) = \begin{array}{c} \hat{y} \\ \hline 0 \quad d \quad d+\delta \end{array} x$$

On peut construire un perceptron multicouche approchant f , pour n'importe quelle précision

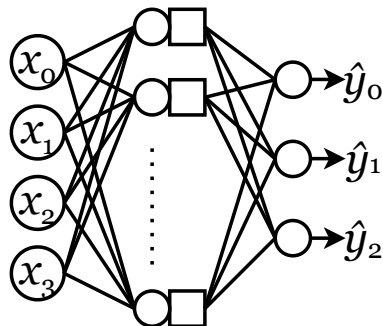
Perceptron pour la classification multiclasse et la régression multiple

Classification :



$$\hat{y} = i^* \text{ t.q. } z_{i^*} = \max_i z_i$$

Régression :



Théorème d'approximation universelle

Neural Networks, Vol. 2, pp. 359–366, 1989
Printed in the USA. All rights reserved.

0893-6080/89 \$3.00 + .00
Copyright © 1989 Pergamon Press plc

ORIGINAL CONTRIBUTION

Multilayer Feedforward Networks are Universal Approximators

Cybenko (1989)
Hornik et al. (1989)

KURT HORNIK

Technische Universität Wien

MAXWELL STINCHCOMBE AND HALBERT WHITE

University of California, San Diego

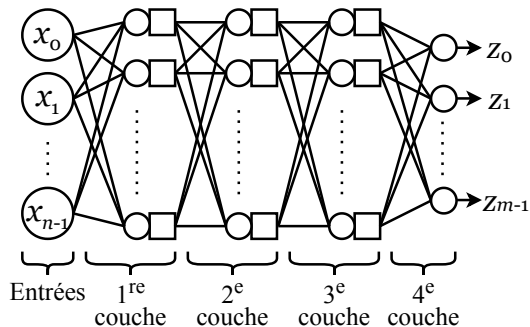
(Received 16 September 1988; revised and accepted 9 March 1989)

Abstract—This paper rigorously establishes that standard multilayer feedforward networks with as few as one hidden layer using arbitrary squashing functions are capable of approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available. In this sense, multilayer feedforward networks are a class of universal approximators.

Keywords—Feedforward networks, Universal approximation, Mapping networks, Network representation capability, Stone-Weierstrass Theorem, Squashing functions, Sigma-Pi networks, Back-propagation networks.

À propos des couches et de la profondeur

- Dans un graphe orienté acyclique, la profondeur se mesure comme la longueur du chemin maximal
- La i -ème couche est composée des neurones de profondeur i
- À partir de trois couches le réseau est considéré « profond »



Suffisance de l'architecture, et besoin de profondeur

Abstract— *This paper rigorously establishes that standard multilayer feedforward networks with as few as one hidden layer using arbitrary squashing functions are capable of approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available. In this sense, multilayer feedforward networks are a class of universal approximators.*

Suffisance de l'architecture, et besoin de profondeur

Abstract— *This paper rigorously establishes that standard multilayer feedforward networks with as few as one hidden layer using arbitrary squashing functions are capable of approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, **provided sufficiently many hidden units are available**. In this sense, multilayer feedforward networks are a class of universal approximators.*

- L'expressivité du réseau est contrainte par le nombre de neurones

Suffisance de l'architecture, et besoin de profondeur

Abstract— *This paper rigorously establishes that standard multilayer feedforward networks with as few as one hidden layer using arbitrary squashing functions are capable of approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available. In this sense, multilayer feedforward networks are a class of universal approximators.*

- L'expressivité du réseau est contrainte par le nombre de neurones
- Une architecture avec une seule couche cachée est en théorie suffisante
Mais le nombre de neurones requis est potentiellement illimité ! En général, les réseaux profonds nécessitent moins de neurones pour la même précision.

Suffisance de l'architecture, et besoin de profondeur

Abstract— *This paper rigorously establishes that standard multilayer feedforward networks with as few as one hidden layer **using arbitrary squashing functions** are capable of approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available. In this sense, multilayer feedforward networks are a class of universal approximators.*

- L'expressivité du réseau est contrainte par le nombre de neurones
- Une architecture avec une seule couche cachée est en théorie suffisante
Mais le nombre de neurones requis est potentiellement illimité ! En général, les réseaux profonds nécessitent moins de neurones pour la même précision.
- Le choix de la fonction d'activation n'est pas anodin (sans être restreint aux seules « fonctions écrasantes »)

Suffisance de l'architecture, et besoin de profondeur

Abstract— *This paper rigorously establishes that standard multilayer feedforward networks with as few as one hidden layer using arbitrary squashing functions **are capable of** approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available. In this sense, multilayer feedforward networks are a class of universal approximators.*

- L'expressivité du réseau est contrainte par le nombre de neurones
- Une architecture avec une seule couche cachée est en théorie suffisante
Mais le nombre de neurones requis est potentiellement illimité ! En général, les réseaux profonds nécessitent moins de neurones pour la même précision.
- Le choix de la fonction d'activation n'est pas anodin (sans être restreint aux seules « fonctions écrasantes »)
- Un petit détail...

Suffisance de l'architecture, et besoin de profondeur

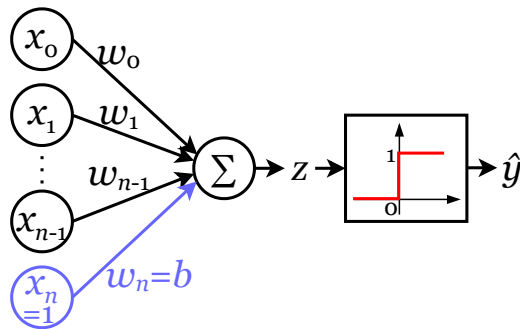
Abstract— *This paper rigorously establishes that standard multilayer feedforward networks with as few as one hidden layer using arbitrary squashing functions **are capable of** approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available. In this sense, multilayer feedforward networks are a class of universal approximators.*

- L'expressivité du réseau est contrainte par le nombre de neurones
- Une architecture avec une seule couche cachée est en théorie suffisante
Mais le nombre de neurones requis est potentiellement illimité ! En général, les réseaux profonds nécessitent moins de neurones pour la même précision.
- Le choix de la fonction d'activation n'est pas anodin (sans être restreint aux seules « fonctions écrasantes »)
- Un petit détail... Il s'agit d'un théorème d'existence

Apprendre avec un réseau de neurones, historique

- McCulloch and Pitts (1943) :
Pas de mécanisme d'apprentissage
- Rosenblatt (1958)
Algorithme du Perceptron

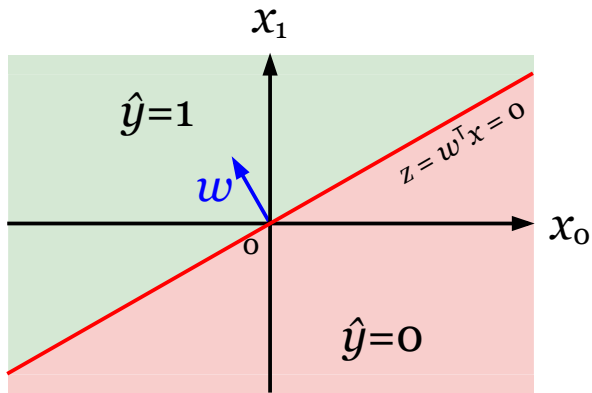
Le Perceptron, redéfinition avec le biais comme poids



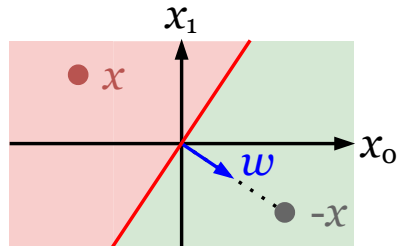
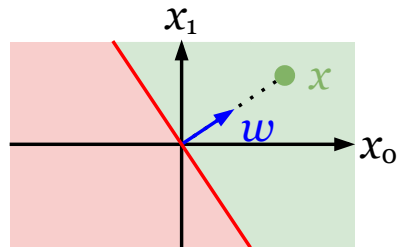
$$z = \sum_{i=0}^n w_i x_i = \sum_{i=0}^{n-1} w_i x_i + b$$
$$\hat{y} = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{sinon} \end{cases}$$

↪ Ajout d'une coordonnée 1 pour tout x : tous les paramètres du réseau sont contenus dans w

Le vecteur poids dans l'espace des entrées, retour du gabarit



$\hookrightarrow w$ définit l'hyperplan séparateur



Algorithme 1 Perceptron

Entrée :

- $\mathcal{S}_{\text{train}}$: jeu d'apprentissage de classification binaire ($\mathcal{X} = \mathbb{R}^n$, $\mathcal{Y} = \{0, 1\}$)
- α : taux d'apprentissage

Sortie : un modèle linéaire entraîné pour la tâche décrite par $\mathcal{S}_{\text{train}}$

fonction apprendre($\mathcal{S}_{\text{train}}, \alpha$)

construire $\mathcal{S}'_{\text{train}}$ en ajoutant une coordonnée 1 à x , pour chaque exemple $(x, y) \in \mathcal{S}_{\text{train}}$

initialiser arbitrairement $w \in \mathbb{R}^{n+1}$

$t \leftarrow 1$

répéter

pour tout exemple $(x, y) \in \mathcal{S}'_{\text{train}}$ **faire**

$\hat{y} \leftarrow f(w^\top x)$

si $\hat{y} \neq y$ **alors** /* le perceptron commet une erreur */
/* t-ième m-à-j des paramètres */

si $y = 1$ **alors**

$w \leftarrow w + \alpha x$

sinon /* $y = 0$ */

$w \leftarrow w - \alpha x$

fin si

$t \leftarrow t + 1$

fin si

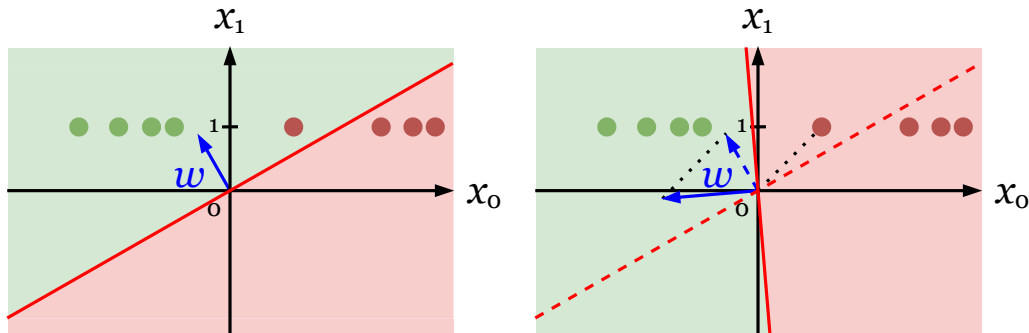
fin pour

jusqu'à w sépare sans erreur $\mathcal{S}'_{\text{train}}$ **ou** un critère d'arrêt est vérifié

retourner w

fin fonction

Algorithme du Perceptron, m-à-j de w quand le modèle fait une erreur



↪ Erreur pour un exemple de classe négative ($y = 0, \hat{y} = 1$) : soustraire x à w

Apprendre avec un réseau de neurones, historique

- McCulloch and Pitts (1943) :
Pas de mécanisme d'apprentissage
- Rosenblatt (1958)
Algorithme du Perceptron

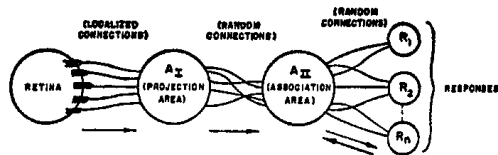
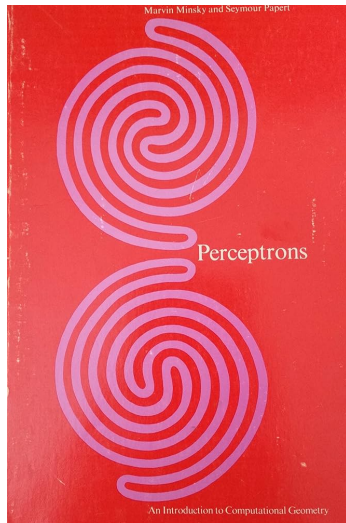


FIG. 1. Organization of a perceptron.

Apprendre avec un réseau de neurones, historique

- McCulloch and Pitts (1943) :
Pas de mécanisme d'apprentissage
- Rosenblatt (1958)
Algorithme du Perceptron
- Minsky and Papert (1969)
Quid du passage à l'échelle ?



Apprendre avec un réseau de neurones, historique

- McCulloch and Pitts (1943) :
Pas de mécanisme d'apprentissage
- Rosenblatt (1958)
Algorithme du Perceptron
- Minsky and Papert (1969)
Quid du passage à l'échelle ?
- Rumelhart et al. (1986)
Implémentation de la rétropropagation
pour les réseaux de neurones

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California,
San Diego, La Jolla, California 92093, USA

† Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors². Learning becomes more interesting but

L'apprentissage supervisé, une question d'optimisation

- θ : vecteur regroupant l'ensemble des paramètres du modèle (poids et biais de tous les neurones du réseau)
- $\mathcal{S}$: un ensemble d'exemples, couples entrée-sortie, décrivant implicitement la fonction, ou tâche, que l'on veut reproduire
- J : une fonction qui estime l'aptitude du modèle à réaliser correctement la tâche voulue (souvent vue comme une mesure de coût)

Objectif : Trouver les paramètres qui permettent de minimiser J

$$\theta^* = \arg \min_{\theta} J(\theta, \mathcal{S})$$

Fonctions objectifs courantes pour les réseaux de neurones

Régression : Erreur quadratique moyenne

$$J_{\text{MSE}}(\theta, \mathcal{S}) = \frac{1}{|\mathcal{S}|} \sum_{(x,y) \in \mathcal{S}} (y - \hat{y})^2$$

où $\hat{y} = f_{\theta}(x)$

Classification : Log-vraisemblance négative

$$J_{\text{NLL}}(\theta, \mathcal{S}) = -\frac{1}{|\mathcal{S}|} \sum_{(x,y) \in \mathcal{S}} \log P_{\theta}(Y = y | X = x)$$

Fonctions objectifs courantes pour les réseaux de neurones

Régression : Erreur quadratique moyenne

$$J_{\text{MSE}}(\theta, \mathcal{S}) = \frac{1}{|\mathcal{S}|} \sum_{(x,y) \in \mathcal{S}} (y - \hat{y})^2$$

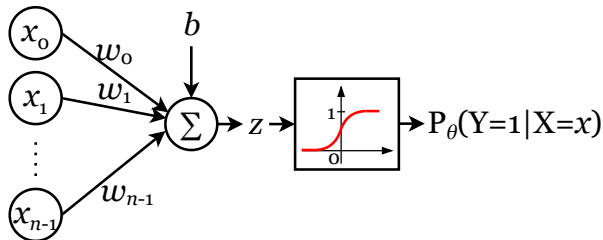
où $\hat{y} = f_{\theta}(x)$

Classification : Log-vraisemblance négative

$$J_{\text{NLL}}(\theta, \mathcal{S}) = -\frac{1}{|\mathcal{S}|} \sum_{(x,y) \in \mathcal{S}} \log P_{\theta}(Y = y | X = x)$$

↪ *Comment calculer $P_{\theta}(Y = y | X = x)$?*

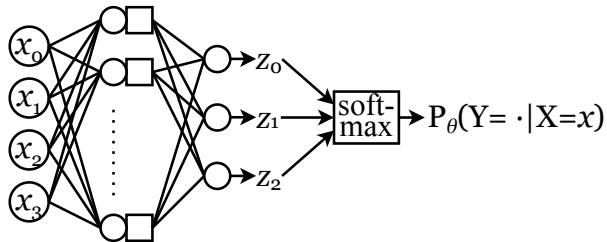
Le Perceptron, version probabiliste



Classification binaire :

calcul de $\sigma(z)$ avec la fonction Sigmoid

$$P_\theta(Y = 1|X = x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

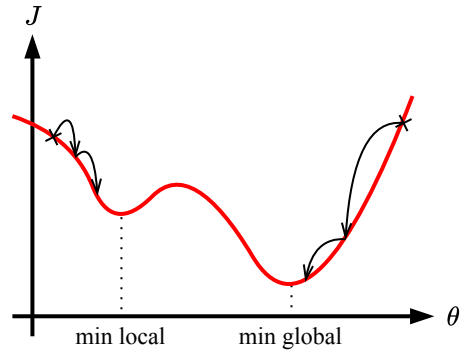
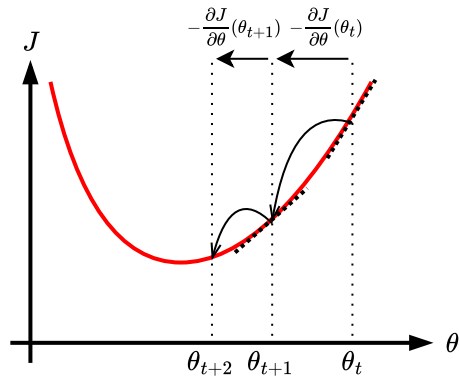


Classification multiclasse :

calcul de $\sigma(z)$ avec la fonction Softmax

$$P_\theta(Y = i|X = x) = (\sigma(z))_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Optimiser les paramètres par descente de gradient



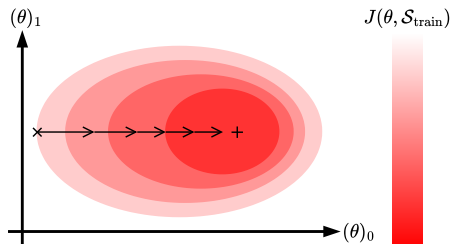
Mise à jour des paramètres, en suivant la direction selon laquelle la fonction de coût diminue :

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial J}{\partial \theta}(\theta_t, \mathcal{S}_t)$$

Descente de gradient, entre rapidité et stabilité

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial J}{\partial \theta}(\theta_t, \mathcal{S}_t)$$

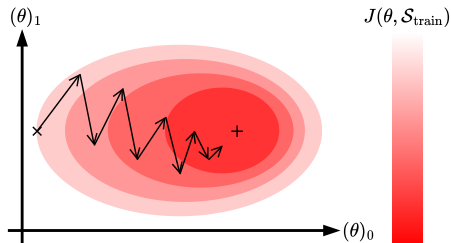
- Si $\mathcal{S}_t = \mathcal{S}_{\text{train}}$ le jeu d'apprentissage entier : *Batch Gradient Descent*.



Descente de gradient, entre rapidité et stabilité

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial J}{\partial \theta}(\theta_t, \mathcal{S}_t)$$

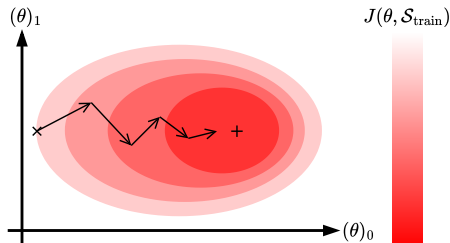
- Si $\mathcal{S}_t = \mathcal{S}_{\text{train}}$ le jeu d'apprentissage entier : *Batch Gradient Descent*.
- Si $\mathcal{S}_t = \{(x, y)\}$ un seul exemple : *Stochastic Gradient Descent*.



Descente de gradient, entre rapidité et stabilité

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial J}{\partial \theta}(\theta_t, \mathcal{S}_t)$$

- Si $\mathcal{S}_t = \mathcal{S}_{\text{train}}$ le jeu d'apprentissage entier : *Batch Gradient Descent*.
- Si $\mathcal{S}_t = \{(x, y)\}$ un seul exemple : *Stochastic Gradient Descent*.
- Si $\mathcal{S}_t$ un petit lot d'exemples : *Mini Batch Gradient Descent*.



Algorithme 2 Apprentissage supervisé par descente de gradient

Entrée :

- $S_{\text{train}}, S_{\text{val}}$: jeux d'apprentissage et de validation, respectivement
- n_{epochs} : nombre d'époques (maximum)
- α : taux d'apprentissage
- J : fonction de coût

Sortie : un modèle entraîné pour la tâche (implicitement) décrite par S_{train} et S_{val}

fonction apprendre($S_{\text{train}}, S_{\text{val}}, n_{\text{epochs}}, \alpha, J$)

initialiser aléatoirement θ

$t \leftarrow 1$

$J_{\text{val}, \min} \leftarrow +\infty$

pour $i \leftarrow 1$ à n_{epochs} **faire**

/* i -ème époque */

pour tout lot S issu de S_{train} **faire**

/* t -ième m-à-j des paramètres

(en pratique, calcul du gradient par rétropropagation) */

$\theta \leftarrow \theta - \alpha \frac{\partial J}{\partial \theta}(\theta, S)$

$t \leftarrow t + 1$

fin pour

/* validation */

$J_{\text{val}} \leftarrow J(\theta, S_{\text{val}})$

si $J_{\text{val}} < J_{\text{val}, \min}$ **alors**

$J_{\text{val}, \min} \leftarrow J_{\text{val}}$

$\theta^* \leftarrow \theta$

sinon

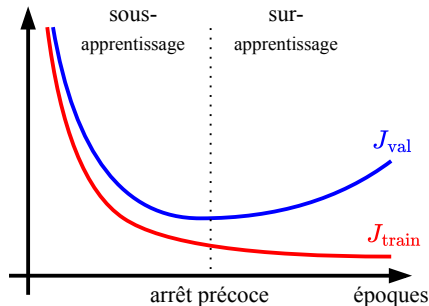
rompre /* arrêt précoce, pour éviter le surapprentissage */

fin si

fin pour

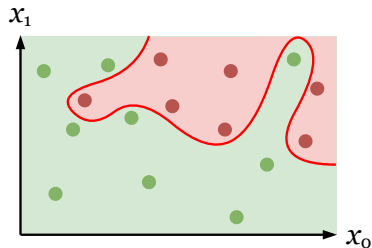
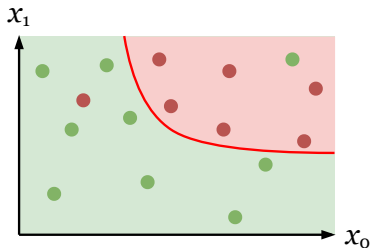
retourner θ^*

fin fonction

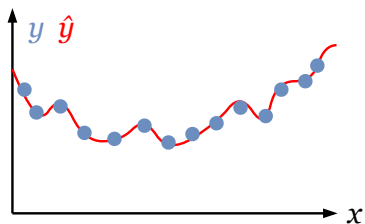
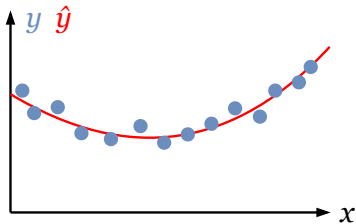


Surapprentissage : sensibilité au bruit et perte de la capacité à généraliser

Classification :



Régression :



Bibliographie I

- Alexander Bain. *Mind and body : The theories of their relation*, volume 4. HS King & Company 1873 Farnborough Eng. Gregg International, 1873.
- Alexandre Bérard. *Neural Machine Translation Architectures and Applications*. Theses, Université de lille, June 2018.
- G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4) :303–314, December 1989.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5) :359–366, 1989.
- Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4) :115–133, Dec 1943.
- Marvin Minsky and Seymour A. Papert. *Perceptrons : An Introduction to Computational Geometry*. The MIT Press, 1969.
- Bhiksha Raj. Introduction to neural networks. Lecture, Lisbon Machine Learning School, July 2023.

Bibliographie II

Frank Rosenblatt. The perceptron : A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6) :386–408, 1958.

David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088) :533–536, 1986.